Terminal ×

```
codio@kiwivatican-brownsunday:~/workspace$
```

Guide ×

Collapse

Pixel Art Module 2

Displaying Pixels

Before continuing, change into the `pixel_art` directory and activate the virtual environment.

```
cd pixel_art/pixel_art
source pa-env/bin/activate
```

Add the following methods to the `Image` class. The `display()` method is going to draw everything to the terminal. Iterate over the `img` attribute. Take each inner-list (`row`) and pass it to the `display_row()` method.

```
  def display(self):
    for row in self.img:
      self.display_row(row)
```

In Python, if you print a list it will print the square brackets with each element separated by a comma. We want just the pixels. Moreover, we want each pixel in the list to appear next to one another. As you iterate over the list, print the pixel **without** the newline character (`end=''`). After the loop runs, print a newline character. This way you are ready for the next row of pixels.

```
  def display_row(self, row):
    for pixel in row:
      print(pixel, end='')
    print()
```

Converting Colors

The `convert_color()` method takes a string that represents a color and returns the associated Colorama object. If the string is not found in `COLORS_DICT` then return a black pixel. `convert_color()` is a helper method for adding pixels to the image.

```
  def convert_color(self, str_clr):
    return COLORS_DICT.get(str_clr, BLACK)
```

Adding Pixels

The last method in the `Image` class is `add_row()`. This method takes an integer as the first parameter. This represents the row of pixels (first row, second row, etc.). After the integer comes an indefinite number of strings. That is why `*args` is used as the parameter; we don't know how many parameters there will be. If the `width` attribute is 7, then there should be eight parameters. The first is the integer designating the row, followed by seven strings for the colors of the pixels.

If the user wants to add a row of pixels all with the same color, it would not be a good user experience if we made them type the same color again and again. If there are only two elements in `args` then the user passed an integer and a string. Use a comprehension to create a Colorama object with `convert_color()`. The comprehension should iterate as many times as the `width` attribute.

```
  def add_row(self, *args):
    if len(args) == 2:
      colors = [self.convert_color(args[1]) for i in range(self.width)]
      self.img[args[0]] = colors
```

If more than two parameters are passed, separate all of the colors from the integer by slicing `args` from the first element to the end of the list. Using a comprehension, transform each color string into its Colorama object with the `convert_color()` method. Finally, use the integer (`args[0]`) to update the row with the list of colors.

```
    else:
      color_params = args[1:]
      colors = [self.convert_color(arg) for arg in color_params]
      self.img[args[0]] = colors
```

▶ **Code**

Lab Question

Assume the following code:

```
python3 -m venv env
```

What is the name of the virtual environment?

- ○ `venv-env`
- ○ `env-venv`
- ◉ `env`
- ○ `venv`

The correct answer is:

```
env
```

When you look at the command to create a virtual environment, `python3 -m` means you are running a module as a script. `venv` is the module that you are running. This is the command that starts the virtual environment creation process. The final `env` is the name for the virtual environment.

Check It!

Next